

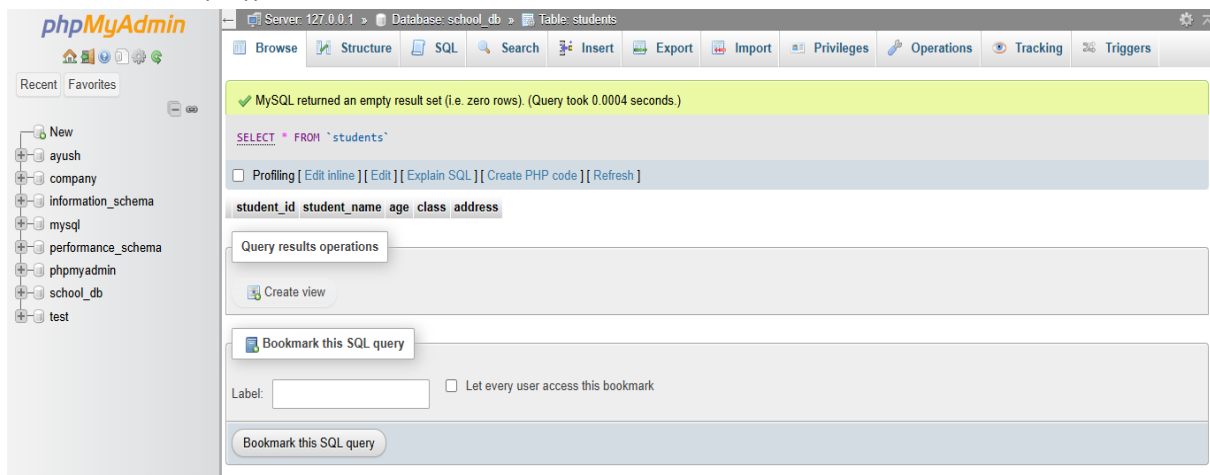
Name : Ayush Hirpara

Module 4 – Introduction to DBMS

1). Introduction to SQL

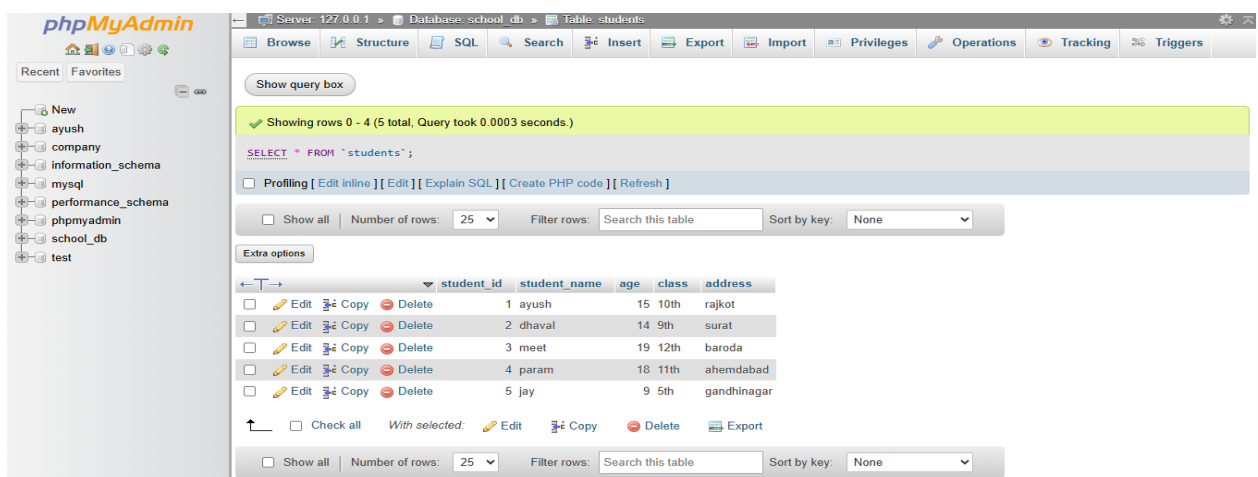
Lab 1 : Create a new database named school_db and a table called students with the following columns: student_id, student_name, age, class, and address.

- CREATE DATABASE school_db
- CREATE TABLE students(student_id int PRIMARY KEY
AUTO_INCREMENT,student_name varchar(20),age int,class varchar(20),address
varchar(20))



Lab 2 : Insert five records into the students table and retrieve all records using the SELECT statement.

- INSERT INTO students(student_name,age,class,address)
VALUES('ayush',15,'10th','rajkot');
- SELECT * FROM `students`



2. SQL Syntax

Lab 1 : Write SQL queries to retrieve specific columns (student_name and age) from the students table.

- SELECT student_name,age FROM students

The screenshot shows a database management tool interface. At the top, it displays 'Server: 127.0.0.1', 'Database: school_db', and 'Table: students'. Below this is a toolbar with icons for Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, and Operations. A 'Show query box' button is visible. The main area shows a green status bar indicating 'Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)'. Below this, the SQL query 'SELECT student_name,age FROM students;' is displayed. There are links for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. Below the query, there are controls for 'Show all', 'Number of rows' (set to 25), 'Filter rows' (Search this table), and 'Sort by key' (None). An 'Extra options' button is also present. The results are shown in a table with columns 'student_name' and 'age'. The data rows are: ayush (15), dhaval (14), meet (19), param (18), and jay (9). Each row has Edit, Copy, and Delete icons. At the bottom, there are 'Check all', 'With selected' (Edit, Copy, Delete, Export), and another set of controls for 'Show all', 'Number of rows' (25), 'Filter rows' (Search this table), and 'Sort by key' (None).

	student_name	age
☐	ayush	15
☐	dhaval	14
☐	meet	19
☐	param	18
☐	jay	9

Lab 2 : Write SQL queries to retrieve all students whose age is greater than 10.

- SELECT * FROM `students` WHERE age>10

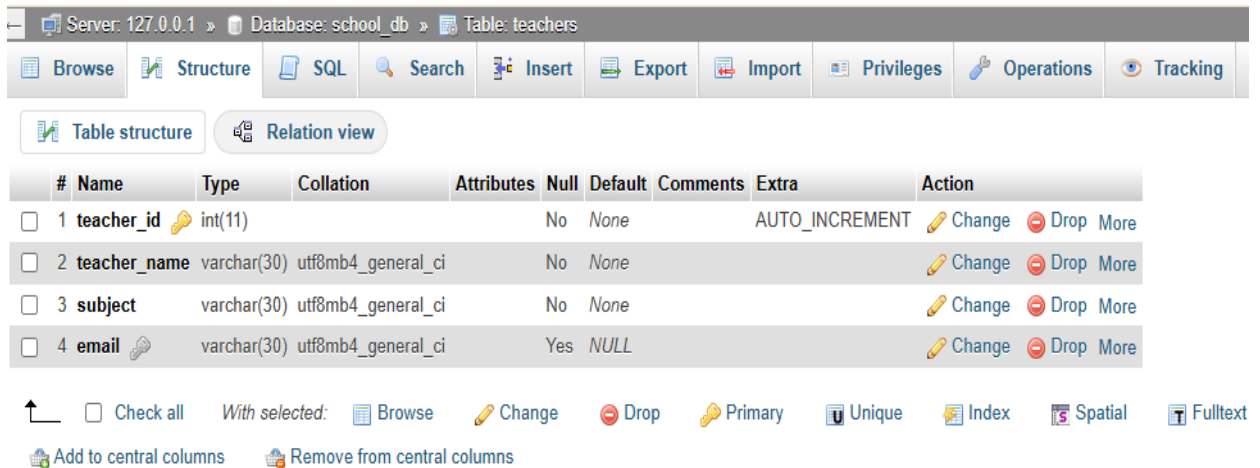
The screenshot shows the same database management tool interface. The SQL query is now 'SELECT * FROM `students` WHERE age>10;'. The status bar indicates 'Showing rows 0 - 3 (4 total, Query took 0.0005 seconds.)'. The results table has columns 'student_id', 'student_name', 'age', 'class', and 'address'. The data rows are: 1 ayush (15, 10th, rajkot), 2 dhaval (14, 9th, surat), 3 meet (19, 12th, baroda), and 4 param (18, 11th, ahemdabad). Each row has Edit, Copy, and Delete icons. The controls at the bottom are the same as in the previous screenshot.

	student_id	student_name	age	class	address
☐	1	ayush	15	10th	rajkot
☐	2	dhaval	14	9th	surat
☐	3	meet	19	12th	baroda
☐	4	param	18	11th	ahemdabad

3. SQL Constraints

Lab 1 : Create a table teachers with the following columns: teacher_id (Primary Key), teacher_name (NOT NULL), subject (NOT NULL), and email (UNIQUE).

- CREATE TABLE teachers (teacher_id int PRIMARY KEY
AUTO_INCREMENT, teacher_name varchar(30) NOT NULL, subject varchar(30) NOT NULL, email varchar(30) UNIQUE)

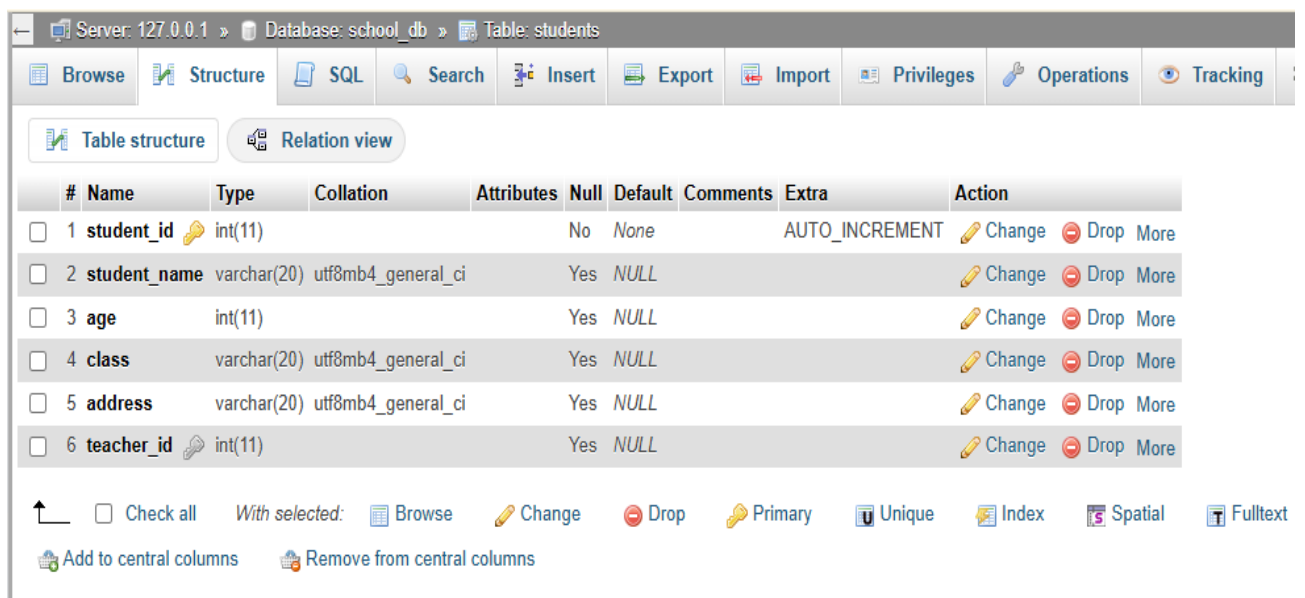


The screenshot shows the phpMyAdmin interface for the 'school_db' database, specifically the 'Table: teachers' view. The 'Table structure' tab is active, displaying a table with 4 columns: teacher_id, teacher_name, subject, and email. The teacher_id column is the primary key and has the AUTO_INCREMENT attribute. The teacher_name, subject, and email columns are NOT NULL. The email column has the UNIQUE attribute. The interface includes a toolbar with options like Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, and Tracking. Below the table structure, there are checkboxes for 'Check all' and 'With selected:', and buttons for 'Browse', 'Change', 'Drop', 'Primary', 'Unique', 'Index', 'Spatial', and 'Fulltext'. There are also buttons for 'Add to central columns' and 'Remove from central columns'.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 teacher_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 teacher_name	varchar(30)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 subject	varchar(30)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 email	varchar(30)	utf8mb4_general_ci		Yes	NULL			Change Drop More

Lab 2 : Implement a FOREIGN KEY constraint to relate the teacher_id from the teachers table with the students table.

- ALTER TABLE students ADD CONSTRAINT fk_students_teacher FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id);



The screenshot shows the phpMyAdmin interface for the 'school_db' database, specifically the 'Table: students' view. The 'Table structure' tab is active, displaying a table with 6 columns: student_id, student_name, age, class, address, and teacher_id. The student_id column is the primary key and has the AUTO_INCREMENT attribute. The student_name, age, class, address, and teacher_id columns are NOT NULL. The teacher_id column has the UNIQUE attribute. The interface includes a toolbar with options like Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, and Tracking. Below the table structure, there are checkboxes for 'Check all' and 'With selected:', and buttons for 'Browse', 'Change', 'Drop', 'Primary', 'Unique', 'Index', 'Spatial', and 'Fulltext'. There are also buttons for 'Add to central columns' and 'Remove from central columns'.

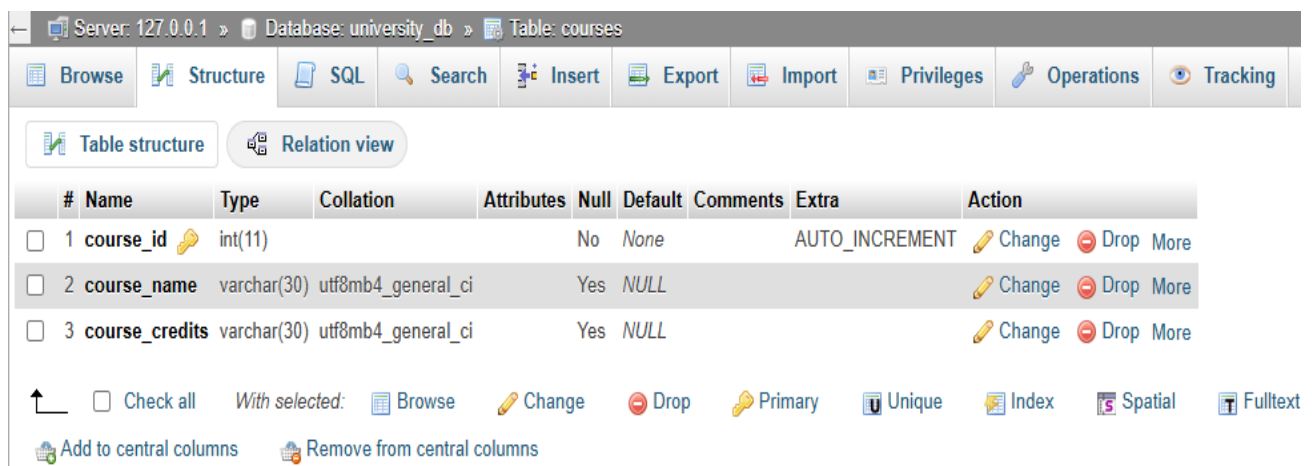
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 student_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 student_name	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	3 age	int(11)			Yes	NULL			Change Drop More
☐	4 class	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	5 address	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	6 teacher_id	int(11)			Yes	NULL			Change Drop More

4. Main SQL Commands and Sub-commands (DDL)

Lab 1 : Create a table courses with columns: course_id, course_name, and course_credits. Set the course_id as the primary key.

Lab 2 : Use the CREATE command to create a database university_db.

- CREATE DATABASE university_db
- CREATE TABLE courses(course_id int PRIMARY KEY AUTO_INCREMENT, course_name varchar(30), course_credits varchar(30))



The screenshot shows a database management interface with the following table structure:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 course_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 course_name	varchar(30)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	3 course_credits	varchar(30)	utf8mb4_general_ci		Yes	NULL			Change Drop More

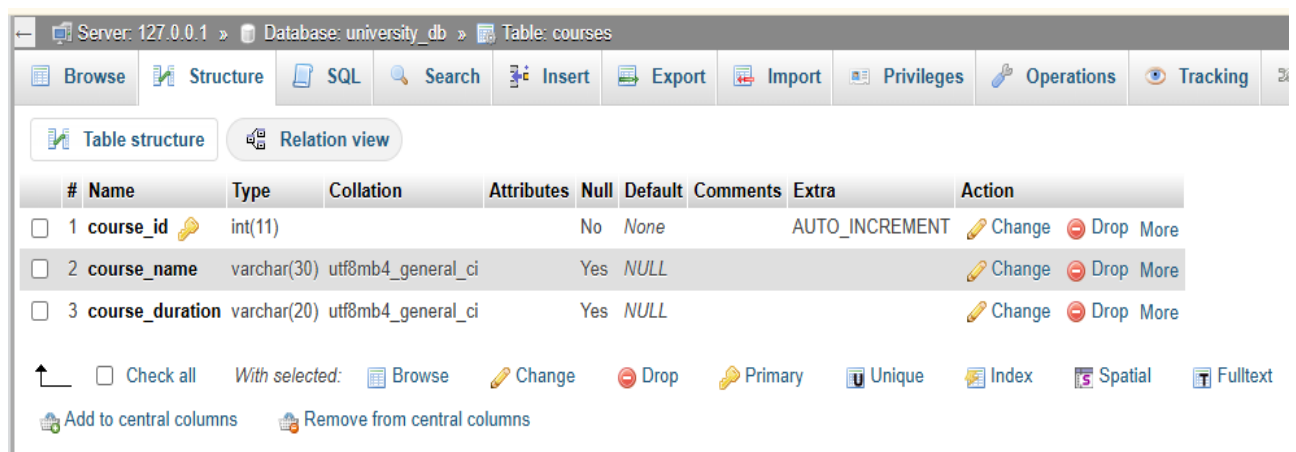
Below the table, there are options to 'Check all', 'With selected', and various actions like 'Browse', 'Change', 'Drop', 'Primary', 'Unique', 'Index', 'Spatial', and 'Fulltext'. There are also buttons to 'Add to central columns' and 'Remove from central columns'.

5. ALTER Command

Lab 1 : Modify the courses table by adding a column course_duration using the ALTER command.

Lab 2 : Drop the course_credits column from the courses table.

- ALTER TABLE courses ADD course_duration varchar(20)
- ALTER TABLE courses DROP course_credits



The screenshot shows the updated table structure after the ALTER command:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 course_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 course_name	varchar(30)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	3 course_duration	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More

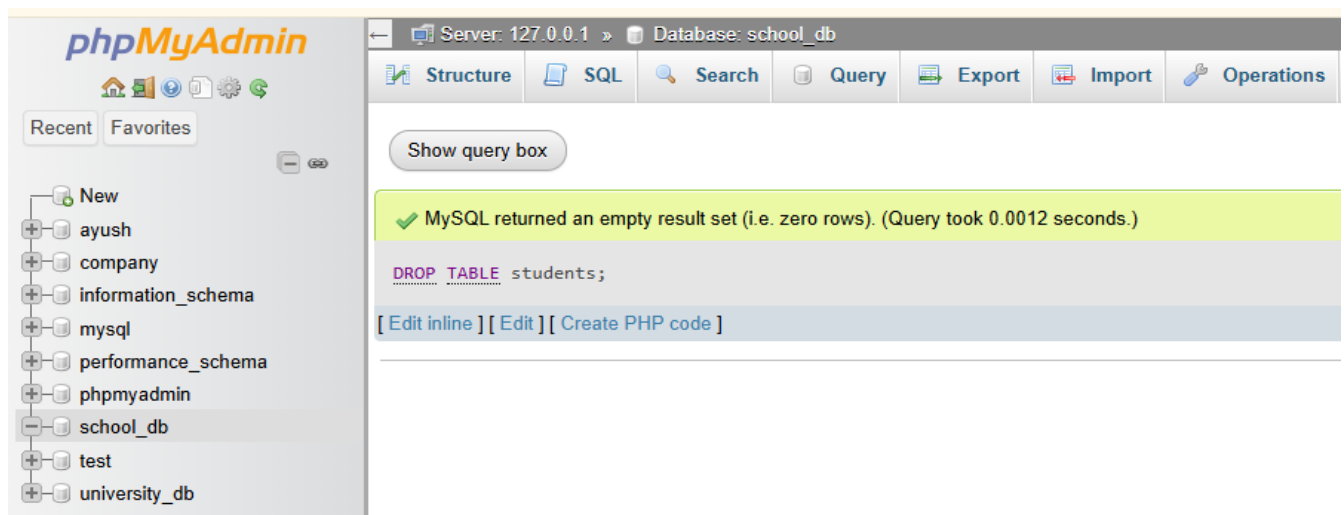
The 'course_credits' column has been removed, and 'course_duration' has been added. The interface also shows the same set of action buttons as in the previous screenshot.

6. DROP Command

Lab 1 : Drop the teachers table from the school_db database.

Lab 2 : Drop the students table from the school_db database and verify that the table has been removed.

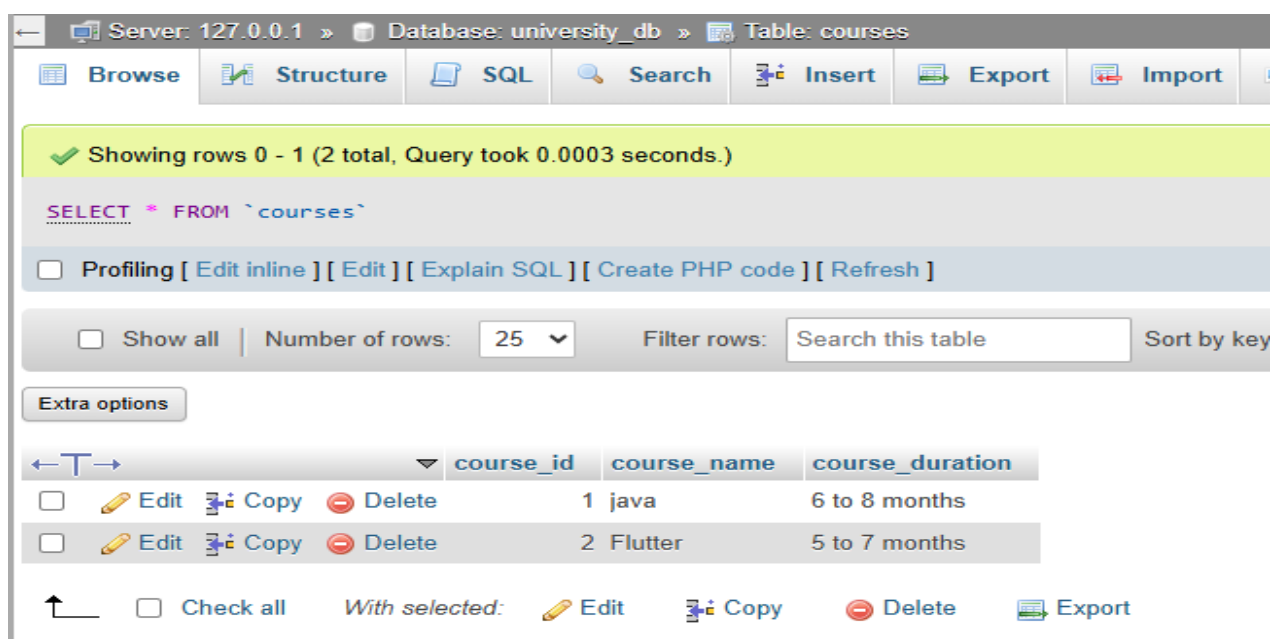
- DROP TABLE teachers
- DROP TABLE students



7. Data Manipulation Language (DML)

Lab 1 : Insert three records into the courses table using the INSERT command.

- INSERT INTO courses (course_name,course_duration) VALUES ('java','6 to 8 months')



Lab 2 : Update the course duration of a specific course using the UPDATE command.

- UPDATE courses SET course_duration = '7 to 9 months' WHERE course_id = 1

Server: 127.0.0.1 » Database: university_db » Table: courses

Showing rows 0 - 1 (2 total, Query took 0.0003 seconds.)

```
SELECT * FROM `courses`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by k

Extra options

	course_id	course_name	course_duration
☐ Edit Copy Delete	1	java	7 to 9 months
☐ Edit Copy Delete	2	Flutter	5 to 7 months

Check all | With selected: Edit Copy Delete Export

Lab 3 : Delete a course with a specific course_id from the courses table using the DELETE command.

- DELETE FROM courses WHERE course_id = 1

Server: 127.0.0.1 » Database: university_db » Table: courses

Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT * FROM `courses`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

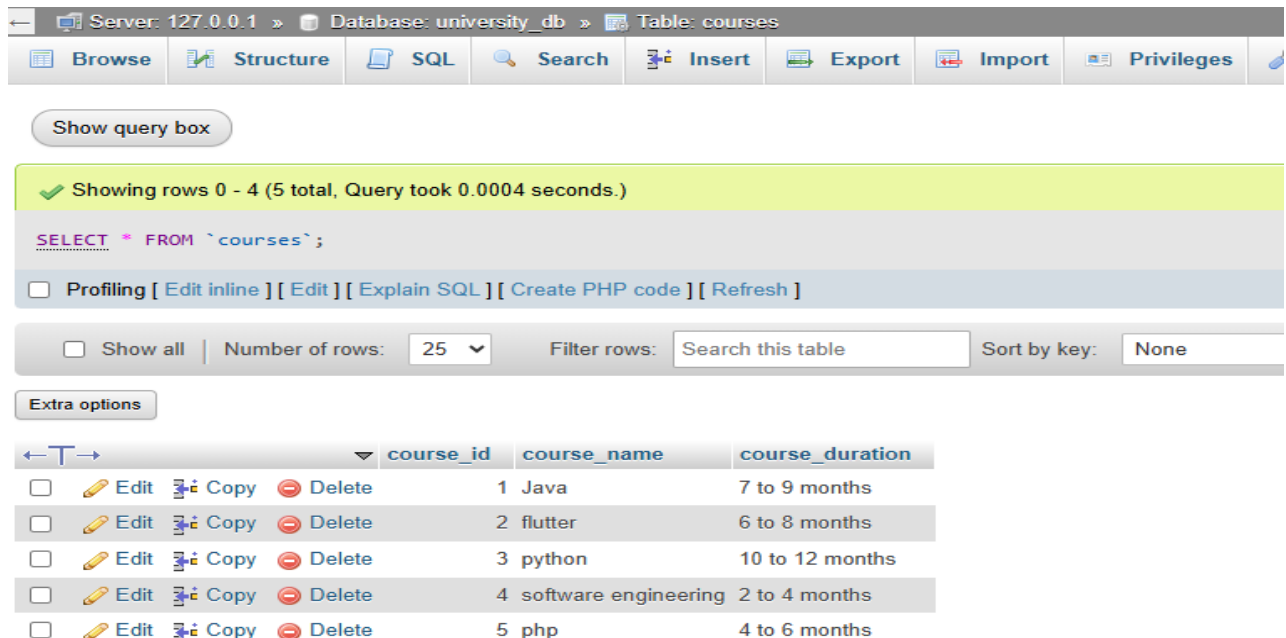
	course_id	course_name	course_duration
☐ Edit Copy Delete	2	Flutter	5 to 7 months

Check all | With selected: Edit Copy Delete Export

8. Data Query Language (DQL)

Lab 1 : Retrieve all courses from the courses table using the SELECT statement.

- `SELECT * FROM `courses``



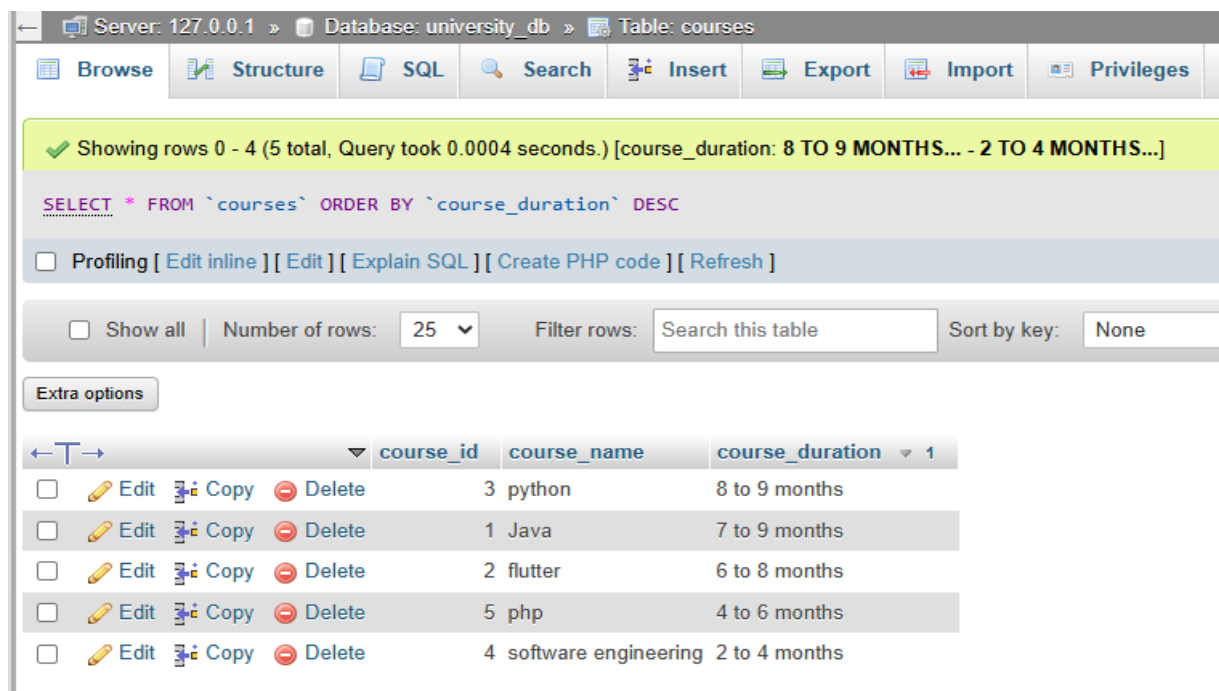
The screenshot shows a database management interface with the following components:

- Header: Server: 127.0.0.1 » Database: university_db » Table: courses
- Navigation bar: Browse, Structure, SQL, Search, Insert, Export, Import, Privileges
- Buttons: Show query box
- Status bar: Showing rows 0 - 4 (5 total, Query took 0.0004 seconds.)
- Query editor: `SELECT * FROM `courses`;`
- Options: Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
- Filters: Show all, Number of rows: 25, Filter rows: Search this table, Sort by key: None
- Extra options button
- Table view:

	course_id	course_name	course_duration
☐ Edit Copy Delete	1	Java	7 to 9 months
☐ Edit Copy Delete	2	flutter	6 to 8 months
☐ Edit Copy Delete	3	python	10 to 12 months
☐ Edit Copy Delete	4	software engineering	2 to 4 months
☐ Edit Copy Delete	5	php	4 to 6 months

Lab 2 : Sort the courses based on course_duration in descending order using ORDER BY.

- `SELECT * FROM `courses` ORDER BY course_duration DESC`



The screenshot shows the same database management interface as before, but with the results sorted by course_duration in descending order. The query editor shows: `SELECT * FROM `courses` ORDER BY `course_duration` DESC`

	course_id	course_name	course_duration
☐ Edit Copy Delete	3	python	8 to 9 months
☐ Edit Copy Delete	1	Java	7 to 9 months
☐ Edit Copy Delete	2	flutter	6 to 8 months
☐ Edit Copy Delete	5	php	4 to 6 months
☐ Edit Copy Delete	4	software engineering	2 to 4 months

Lab 3 : Limit the results of the SELECT query to show only the top two courses using LIMIT.

- `SELECT * FROM `courses` ORDER BY course_duration DESC LIMIT 2;`

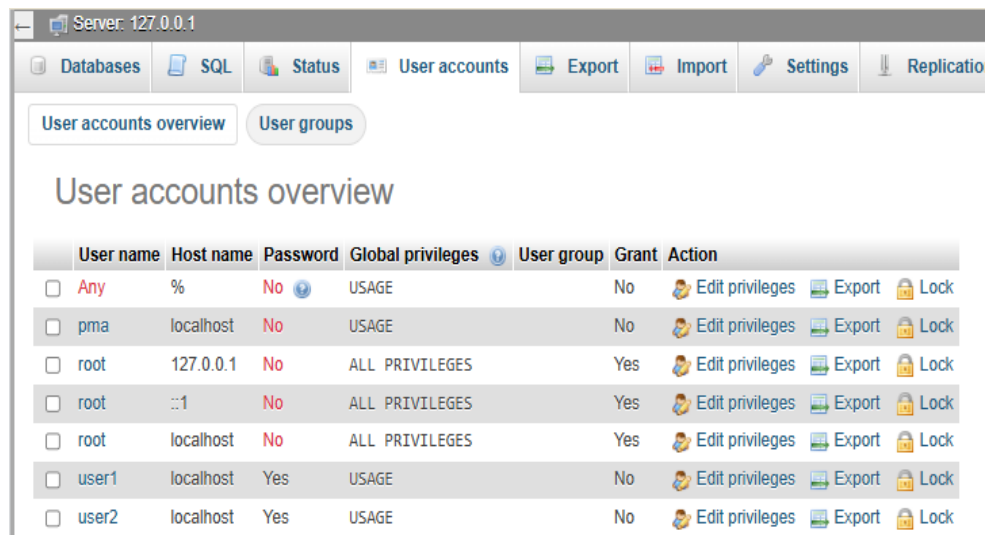
The screenshot shows a web-based database management interface. At the top, there's a breadcrumb trail: "Server: 127.0.0.1 » Database: university_db » Table: courses". Below this is a navigation bar with buttons: "Browse", "Structure", "SQL", "Search", "Insert", "Export", "Import", and "Privileges". A "Show query box" button is also present. The main area displays a green status bar indicating the query results: "Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.) [course_duration: 8 TO 9 MONTHS... - 7 TO 9 MONTHS...]". Below this, the SQL query is shown: `SELECT * FROM `courses` ORDER BY course_duration DESC LIMIT 2;`. There are links for "Profiling", "Edit inline", "Edit", "Explain SQL", "Create PHP code", and "Refresh". An "Extra options" button is also visible. The results are shown in a table with columns: "course_id", "course_name", and "course_duration". The table has two rows: one for "python" (course_id 3) and one for "Java" (course_id 1). Each row has checkboxes and buttons for "Edit", "Copy", and "Delete". At the bottom, there are checkboxes for "Check all" and "With selected:", followed by buttons for "Edit", "Copy", "Delete", and "Export".

	course_id	course_name	course_duration
☐	3	python	8 to 9 months
☐	1	Java	7 to 9 months

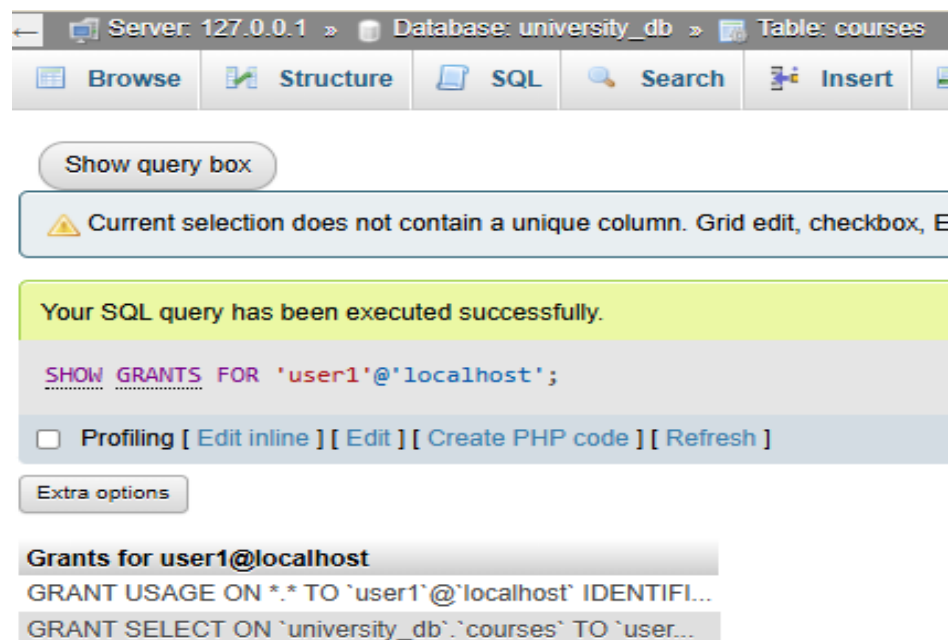
9. Data Control Language (DCL)

Lab 1 : Create two new users user1 and user2 and grant user1 permission to SELECT from the courses table.

- CREATE USER 'user1'@'localhost' IDENTIFIED BY 'user1pass';
- CREATE USER 'user2'@'localhost' IDENTIFIED BY 'user2pass';
- GRANT SELECT ON university_db.courses TO 'user1'@'localhost';



	User name	Host name	Password	Global privileges	User group	Grant	Action
☐	Any	%	No	USAGE		No	Edit privileges Export Lock
☐	pma	localhost	No	USAGE		No	Edit privileges Export Lock
☐	root	127.0.0.1	No	ALL PRIVILEGES	Yes	Yes	Edit privileges Export Lock
☐	root	::1	No	ALL PRIVILEGES	Yes	Yes	Edit privileges Export Lock
☐	root	localhost	No	ALL PRIVILEGES	Yes	Yes	Edit privileges Export Lock
☐	user1	localhost	Yes	USAGE		No	Edit privileges Export Lock
☐	user2	localhost	Yes	USAGE		No	Edit privileges Export Lock



Server: 127.0.0.1 » Database: university_db » Table: courses

Browse Structure SQL Search Insert

Show query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, E

Your SQL query has been executed successfully.

```
SHOW GRANTS FOR 'user1'@'localhost';
```

☐ Profiling [Edit inline] [Edit] [Create PHP code] [Refresh]

Extra options

Grants for user1@localhost

```
GRANT USAGE ON *.* TO 'user1'@'localhost' IDENTIFI...
GRANT SELECT ON `university_db`.`courses` TO `user...
```

Lab 2 : Revoke the INSERT permission from user1 and give it to user2.

- REVOKE INSERT ON university_db.courses FROM 'user1'@'localhost';
- GRANT INSERT ON university_db.courses TO 'user2'@'localhost';

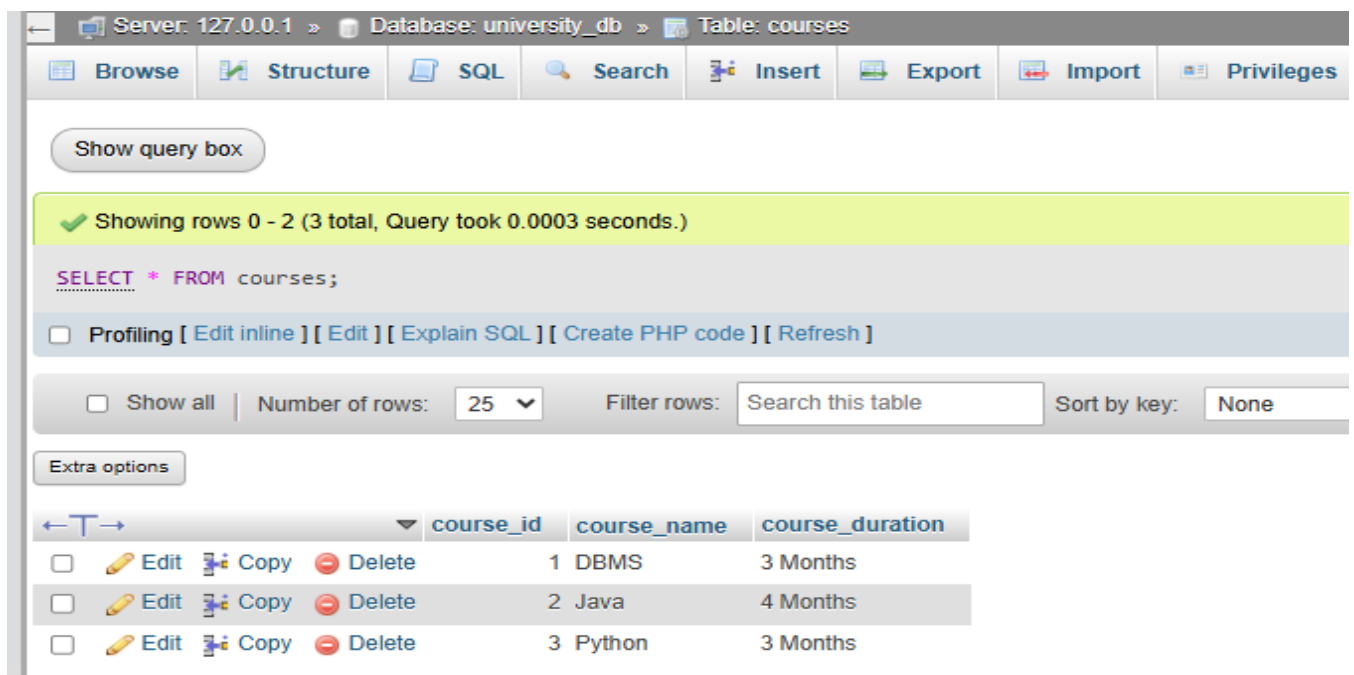
The screenshot shows the phpMyAdmin interface for the 'university_db' database, specifically the 'courses' table. The 'SQL' tab is active. A message states: 'Your SQL query has been executed successfully.' Below this, the executed query is shown: `SHOW GRANTS FOR 'user1'@'localhost';`. Underneath the query, there are links for 'Profiling', 'Edit inline', 'Edit', 'Create PHP code', and 'Refresh'. An 'Extra options' button is also present. The results section, titled 'Grants for user1@localhost', displays two lines of SQL grants: `GRANT USAGE ON *.* TO 'user1'@'localhost' IDENTIFI...` and `GRANT SELECT ON `university_db`.`courses` TO `user...`.

The screenshot shows the phpMyAdmin interface for the 'university_db' database, specifically the 'courses' table. The 'SQL' tab is active. A message states: 'Your SQL query has been executed successfully.' Below this, the executed query is shown: `SHOW GRANTS FOR 'user2'@'localhost';`. Underneath the query, there are links for 'Profiling', 'Edit inline', 'Edit', 'Create PHP code', and 'Refresh'. An 'Extra options' button is also present. The results section, titled 'Grants for user2@localhost', displays two lines of SQL grants: `GRANT USAGE ON *.* TO 'user2'@'localhost' IDENTIFI...` and `GRANT INSERT ON `university_db`.`courses` TO `user...`.

10. Transaction Control Language (TCL)

Lab 1 : Insert a few rows into the courses table and use COMMIT to save the changes.

- INSERT INTO courses (course_name, course_duration) VALUES ('DBMS', '3 Months');
INSERT INTO courses (course_name, course_duration) VALUES ('Java', '4 Months');
INSERT INTO courses (course_name, course_duration) VALUES ('Python', '3 Months');
COMMIT;

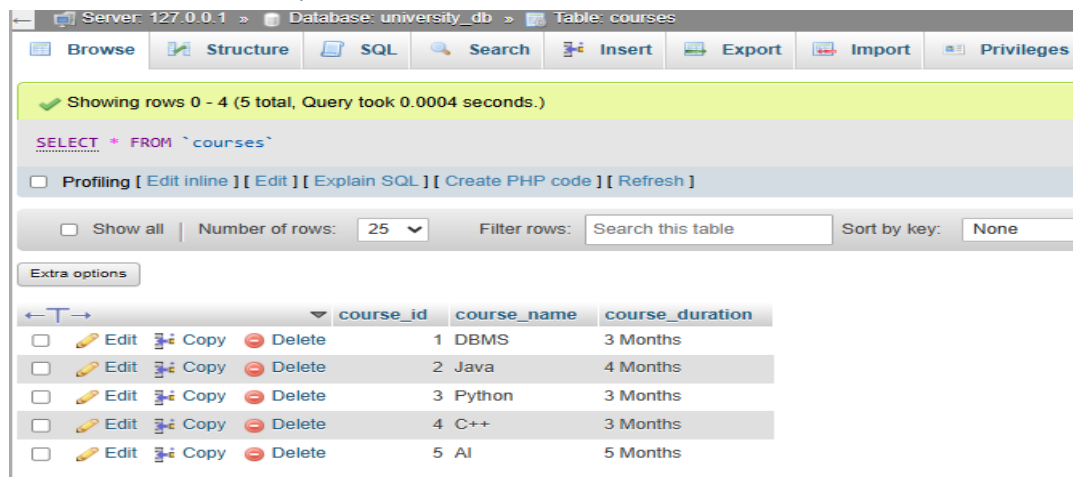


The screenshot shows a database management interface for a server at 127.0.0.1, database 'university_db', and table 'courses'. The interface includes tabs for Browse, Structure, SQL, Search, Insert, Export, Import, and Privileges. A 'Show query box' button is present. A green status bar indicates 'Showing rows 0 - 2 (3 total, Query took 0.0003 seconds.)'. The SQL query 'SELECT * FROM courses;' is entered. Below the query, there are links for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A control bar shows 'Show all', 'Number of rows: 25', 'Filter rows: Search this table', and 'Sort by key: None'. An 'Extra options' button is also visible. The table data is displayed as follows:

	course_id	course_name	course_duration
☐ Edit Copy Delete	1	DBMS	3 Months
☐ Edit Copy Delete	2	Java	4 Months
☐ Edit Copy Delete	3	Python	3 Months

Lab 2 : Insert additional rows, then use ROLLBACK to undo the last insert operation.

- SET autocommit = 0;
INSERT INTO courses (course_id, course_name, course_duration) VALUES (6, 'ML', '4 Months');
INSERT INTO courses (course_id, course_name, course_duration) VALUES (7, 'Data Science', '6 Months');
ROLLBACK;
SET autocommit = 1;

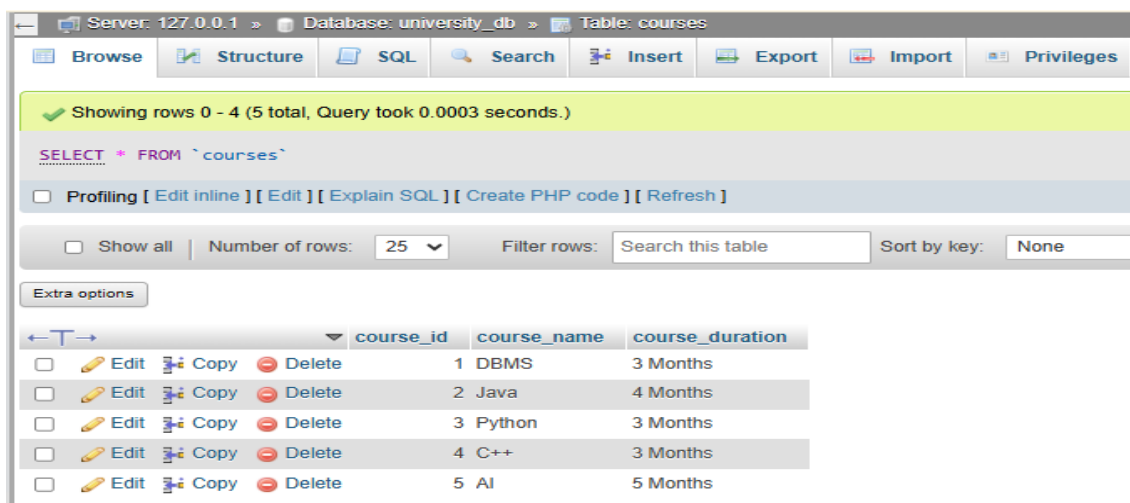


The screenshot shows a database management interface for a table named 'courses' in a database named 'university_db'. The table has three columns: 'course_id', 'course_name', and 'course_duration'. There are 5 rows of data displayed. The interface includes a toolbar with options like 'Browse', 'Structure', 'SQL', 'Search', 'Insert', 'Export', 'Import', and 'Privileges'. A status bar at the top indicates 'Showing rows 0 - 4 (5 total, Query took 0.0004 seconds.)'. Below the table, there are controls for 'Number of rows' (set to 25), 'Filter rows' (a search box), and 'Sort by key' (set to None). The table data is as follows:

course_id	course_name	course_duration
1	DBMS	3 Months
2	Java	4 Months
3	Python	3 Months
4	C++	3 Months
5	AI	5 Months

Lab 3 : Create a SAVEPOINT before updating the courses table, and use it to roll back specific changes.

- START TRANSACTION;
SAVEPOINT before_update;
UPDATE courses SET course_duration = '7 Months' WHERE course_id=5;
ROLLBACK TO before_update;
COMMIT;



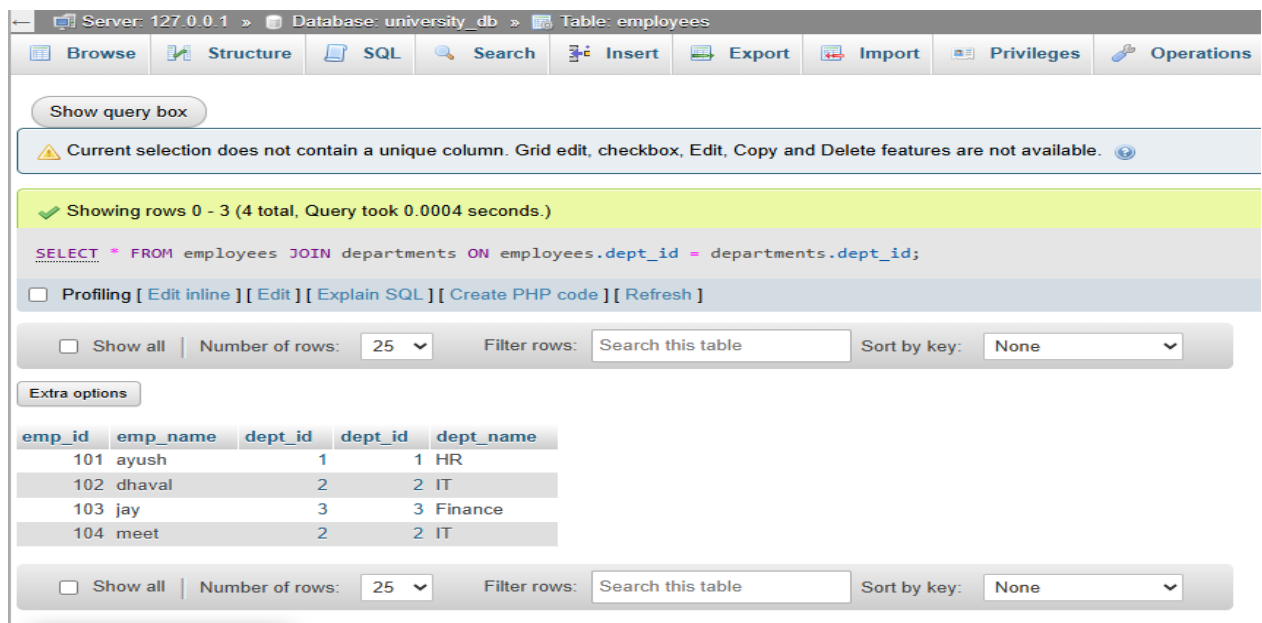
The screenshot shows the same database management interface as before, but with the 'course_duration' for 'course_id' 5 updated to '7 Months'. The status bar at the top indicates 'Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)'. The table data is as follows:

course_id	course_name	course_duration
1	DBMS	3 Months
2	Java	4 Months
3	Python	3 Months
4	C++	3 Months
5	AI	7 Months

11. SQL Joins

Lab 1 : Create two tables: departments and employees. Perform an INNER JOIN to display employees along with their respective departments.

- `SELECT * FROM employees JOIN departments ON employees.dept_id = departments.dept_id;`

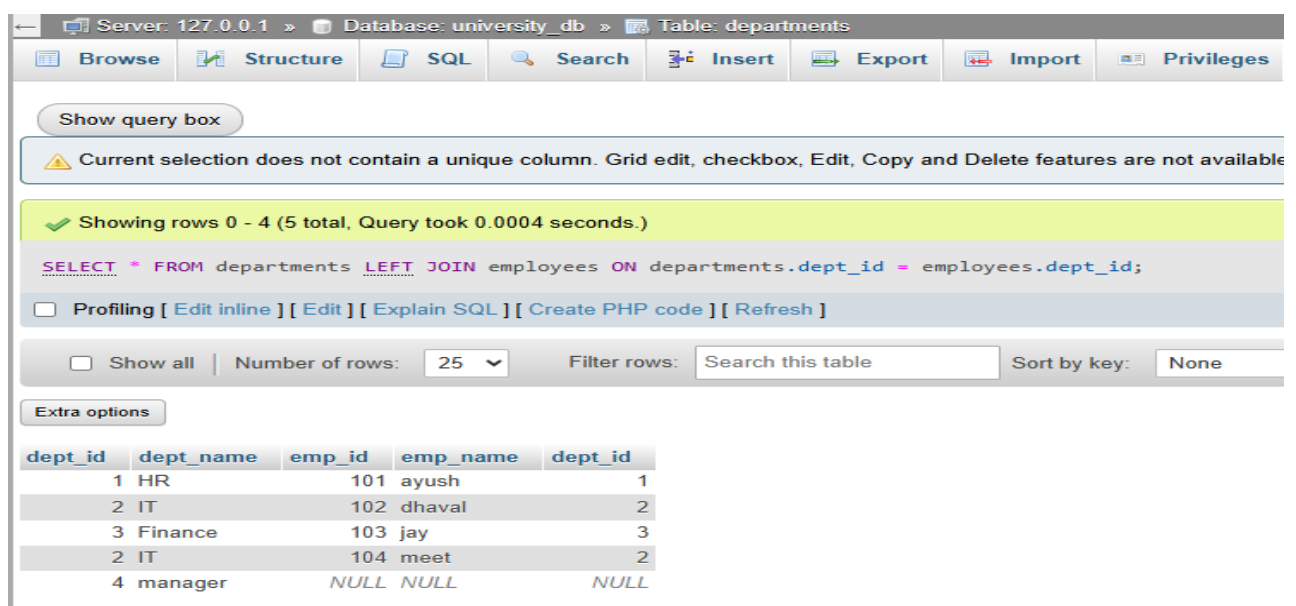


The screenshot shows the phpMyAdmin interface for the 'university_db' database, specifically the 'employees' table. The SQL query executed is `SELECT * FROM employees JOIN departments ON employees.dept_id = departments.dept_id;`. The result shows 4 rows of data, displaying employee details along with their department names. The interface includes a toolbar with options like Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, and Operations. A warning message states: 'Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.' The query execution status is 'Showing rows 0 - 3 (4 total, Query took 0.0004 seconds.)'. Below the query, there are links for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. The result table has columns: emp_id, emp_name, dept_id, dept_id, and dept_name.

emp_id	emp_name	dept_id	dept_id	dept_name
101	ayush	1	1	HR
102	dhaval	2	2	IT
103	jay	3	3	Finance
104	meet	2	2	IT

Lab 2 : Use a LEFT JOIN to show all departments, even those without employees.

- `SELECT * FROM departments LEFT JOIN employees ON departments.dept_id = employees.dept_id;`



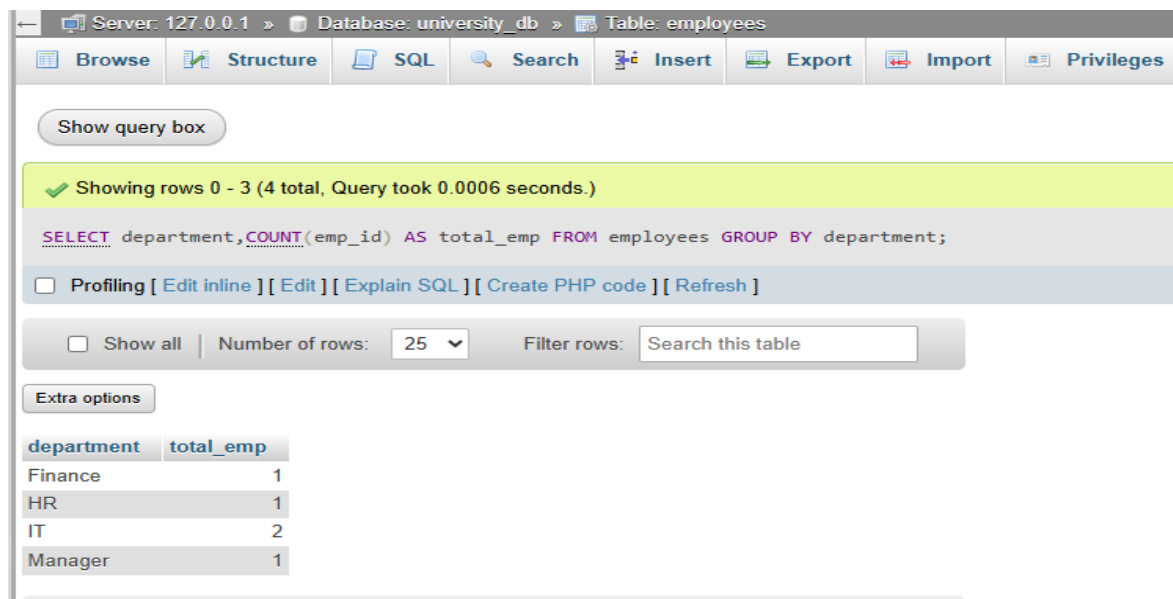
The screenshot shows the phpMyAdmin interface for the 'university_db' database, specifically the 'departments' table. The SQL query executed is `SELECT * FROM departments LEFT JOIN employees ON departments.dept_id = employees.dept_id;`. The result shows 5 rows of data, displaying department details along with employee details where available. The interface includes a toolbar with options like Browse, Structure, SQL, Search, Insert, Export, Import, and Privileges. A warning message states: 'Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.' The query execution status is 'Showing rows 0 - 4 (5 total, Query took 0.0004 seconds.)'. Below the query, there are links for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. The result table has columns: dept_id, dept_name, emp_id, emp_name, and dept_id.

dept_id	dept_name	emp_id	emp_name	dept_id
1	HR	101	ayush	1
2	IT	102	dhaval	2
3	Finance	103	jay	3
2	IT	104	meet	2
4	manager	NULL	NULL	NULL

12. SQL Group By

Lab 1 : Group employees by department and count the number of employees in each department using GROUP BY.

- SELECT department,COUNT(emp_id) AS total_emp FROM employees GROUP BY department



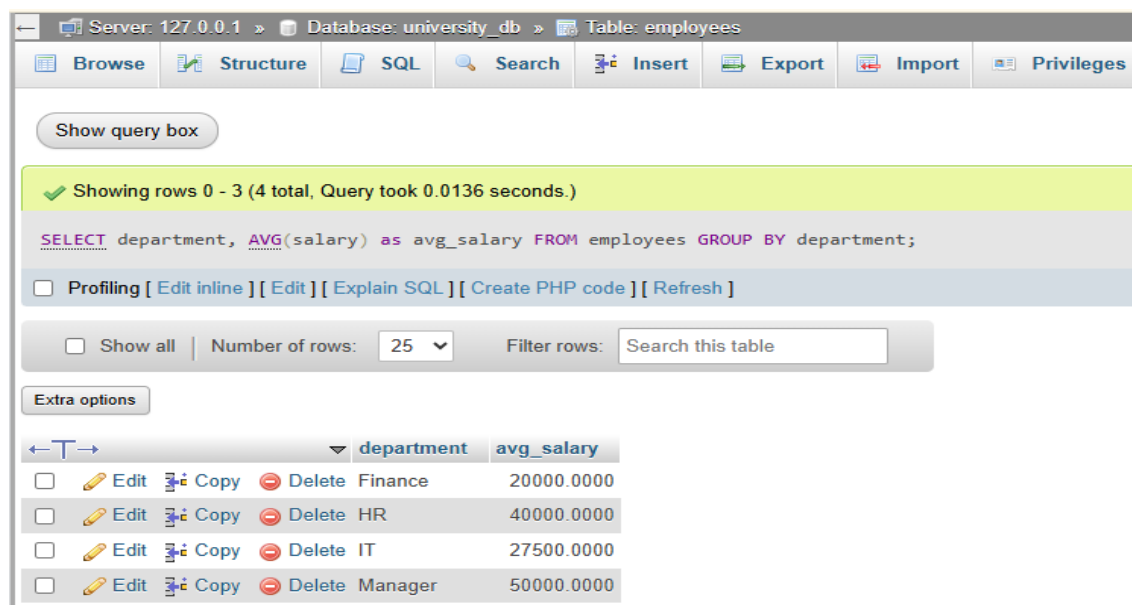
The screenshot shows a database management interface with the following components:

- Header: Server: 127.0.0.1 » Database: university_db » Table: employees
- Navigation tabs: Browse, Structure, SQL, Search, Insert, Export, Import, Privileges
- Buttons: Show query box
- Status bar: Showing rows 0 - 3 (4 total, Query took 0.0006 seconds.)
- SQL query: `SELECT department,COUNT(emp_id) AS total_emp FROM employees GROUP BY department;`
- Options: ☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
- Controls: ☐ Show all | Number of rows: 25 | Filter rows: Search this table
- Extra options button
- Result table:

department	total_emp
Finance	1
HR	1
IT	2
Manager	1

Lab 2 : Use the AVG aggregate function to find the average salary of employees in each department.

- SELECT department, AVG(salary) as avg_salary FROM employees GROUP BY department



The screenshot shows a database management interface with the following components:

- Header: Server: 127.0.0.1 » Database: university_db » Table: employees
- Navigation tabs: Browse, Structure, SQL, Search, Insert, Export, Import, Privileges
- Buttons: Show query box
- Status bar: Showing rows 0 - 3 (4 total, Query took 0.0136 seconds.)
- SQL query: `SELECT department, AVG(salary) as avg_salary FROM employees GROUP BY department;`
- Options: ☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
- Controls: ☐ Show all | Number of rows: 25 | Filter rows: Search this table
- Extra options button
- Result table:

	department	avg_salary
☐ Edit ☐ Copy ☐ Delete	Finance	20000.0000
☐ Edit ☐ Copy ☐ Delete	HR	40000.0000
☐ Edit ☐ Copy ☐ Delete	IT	27500.0000
☐ Edit ☐ Copy ☐ Delete	Manager	50000.0000

13. SQL Stored Procedure

Lab 1 : Write a stored procedure to retrieve all employees from the employees table based on department.

- DELIMITER //

```
CREATE PROCEDURE get_emp_proc (IN dept_name varchar(20))
BEGIN
SELECT * FROM employees WHERE department = dept_name;
END //

DELIMITER ;
```

The screenshot shows a database management tool interface with a toolbar at the top containing icons for Structure, SQL, Search, Query, Export, Import, and Operations. Below the toolbar, a green message box states: "Your SQL query has been executed successfully. 2 rows affected by the last statement inside the procedure." Below this, the SQL code is displayed: `SET @p0='manager'; CALL `get_emp_proc` (@p0);`. A section titled "Execution results of routine `get\_emp\_proc`" contains a table with the following data:

emp_id	emp_name	department	salary
4	meet	Manager	50000
8	suresh	manager	55000

Below the table, there is a "Routines" section with a search icon. It includes buttons for "Check all", "Export", and "Drop". A table lists the routines:

	Name	Type	Returns	
☐	get_emp_proc	PROCEDURE		Edit Execute Export Drop

Lab 2 : Write a stored procedure that accepts course_id as input and returns the course details.

- DELIMITER //

```
CREATE PROCEDURE course_detail_proc (IN c_id int)
BEGIN
SELECT * FROM courses WHERE course_id = c_id;
END //
```

DELIMITER ;

The screenshot shows a database management tool interface. At the top, it displays 'Server: 127.0.0.1' and 'Database: university_db'. Below this is a toolbar with icons for Structure, SQL, Search, Query, Export, Import, and Operations. A green message box states: 'Your SQL query has been executed successfully. 1 row affected by the last statement inside the procedure.' Below this, the SQL code is shown: 'SET @p0='2'; CALL `course\_detail\_proc` (@p0);'. A tooltip 'Execution results of routine `course\_detail\_proc`' is displayed over a table with the following data:

course_id	course_name	course_duration
2	flutter	6 to 8 months

Below the table, the 'Routines' section is visible. It includes buttons for 'Check all', 'Export', and 'Drop'. A table lists the routines:

Name	Type	Returns	Actions
☐ course_detail_proc	PROCEDURE		Edit Execute Export Drop
☐ get_emp_proc	PROCEDURE		Edit Execute Export Drop

14. SQL View

Lab 1 : Create a view to show all employees along with their department names.

- CREATE VIEW emp_with_dept_view AS SELECT emp_name,department FROM employees

The screenshot shows the SQL Studio interface with the view 'emp_with_dept_view' selected. The SQL query is `SELECT * FROM `emp_with_dept_view``. The view displays 8 rows of employee data. The interface includes a toolbar with options like Browse, Structure, SQL, Search, Insert, Export, and Privileges. A warning message states: 'Current selection does not contain a unique column. Grid edit, Edit, Copy and Delete features may result in un'. The status bar indicates 'Showing rows 0 - 7 (8 total, Query took 0.0004 seconds.)'. Below the query, there are links for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. The 'Extra options' section shows a table with columns 'emp_name' and 'department'.

	emp_name	department
☐	ayush	IT
☐	dhaval	HR
☐	jay	Finance
☐	meet	Manager
☐	param	IT
☐	mahesh	HR
☐	ramesh	chemical
☐	suresh	manager

Lab 2 : Modify the view to exclude employees whose salaries are below \$50,000.

- CREATE VIEW emp_salary_view AS SELECT emp_id,emp_name,department,salary FROM employees WHERE salary<=50000

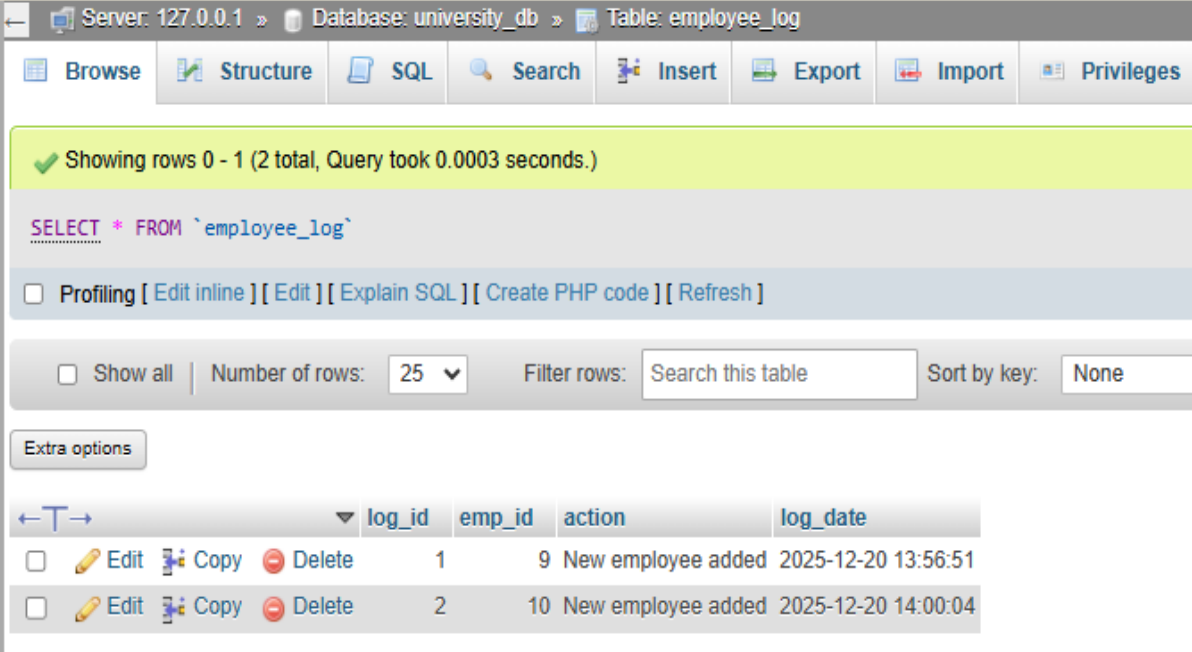
The screenshot shows the SQL Studio interface with the view 'emp_salary_view' selected. The SQL query is `SELECT * FROM `emp_salary_view``. The view displays 4 rows of employee data. The interface includes a toolbar with options like Browse, Structure, SQL, Search, Insert, Export, and Privileges. A warning message states: 'Current selection does not contain a unique column. Grid edit, Edit, Copy and Delete features may result in'. The status bar indicates 'Showing rows 0 - 3 (4 total, Query took 0.0004 seconds.)'. Below the query, there are links for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. The 'Extra options' section shows a table with columns 'emp_id', 'emp_name', 'department', and 'salary'.

	emp_id	emp_name	department	salary
☐	1	ayush	IT	30000
☐	2	dhaval	HR	40000
☐	3	jay	Finance	20000
☐	5	param	IT	25000

15. SQL Triggers

Lab 1 : Create a trigger to automatically log changes to the employees table when a new employee is added.

- DELIMITER \$\$
CREATE TRIGGER after_emp_insert AFTER INSERT ON employees FOR EACH ROW
BEGIN
INSERT INTO employee_log (emp_id, action) VALUES (NEW.emp_id, 'New employee
added');
END \$\$
DELIMITER ;



The screenshot shows a database management interface with the following components:

- Header:** Server: 127.0.0.1 » Database: university_db » Table: employee_log
- Navigation Bar:** Browse, Structure, SQL, Search, Insert, Export, Import, Privileges
- Status Bar:** Showing rows 0 - 1 (2 total, Query took 0.0003 seconds.)
- SQL Editor:** `SELECT * FROM `employee_log``
- Tools:** Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
- Display Options:** Show all, Number of rows: 25, Filter rows: Search this table, Sort by key: None
- Extra options:** (button)
- Table Data:**

	log_id	emp_id	action	log_date
☐ Edit Copy Delete	1	9	New employee added	2025-12-20 13:56:51
☐ Edit Copy Delete	2	10	New employee added	2025-12-20 14:00:04

Lab 2 : Create a trigger to update the last_modified timestamp whenever an employee record is updated.

- CREATE TRIGGER before_emp_update BEFORE UPDATE ON employees FOR EACH ROW
BEGIN
 SET NEW.last_modified = NOW();
END;

Server: 127.0.0.1 » Database: university_db » Table: employees

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)

SELECT * FROM `employees`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: No

Extra options

		emp_id	emp_name	department	salary	last_modified
☐	Edit	1	ayush	IT	30000	2025-12-20 14:17:09
☐	Edit	2	dhaval	HR	40000	2025-12-20 14:17:09
☐	Edit	3	jay	Finance	20000	2025-12-20 14:17:09
☐	Edit	4	meet	Manager	52000	2025-12-20 14:17:09
☐	Edit	5	param	IT	25000	2025-12-20 14:17:09

Server: 127.0.0.1 » Database: university_db » Table: employees

Showing rows 0 - 4 (5 total, Query took 0.0004 seconds.)

SELECT * FROM `employees`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: No

Extra options

		emp_id	emp_name	department	salary	last_modified
☐	Edit	1	ayush	IT	35000	2025-12-20 14:30:12
☐	Edit	2	dhaval	HR	45000	2025-12-20 14:30:12
☐	Edit	3	jay	Finance	25000	2025-12-20 14:30:12
☐	Edit	4	meet	Manager	55000	2025-12-20 14:30:12
☐	Edit	5	tilak	IT	30000	2025-12-20 14:30:12

16. Introduction to PL/SQL

Lab 1 : Write a PL/SQL block to print the total number of employees from the employees table.

```
- DELIMITER $$  
CREATE PROCEDURE total_employees()  
BEGIN  
    SELECT COUNT(*) AS total_employees  
    FROM employees;  
END $$  
DELIMITER ;  
CALL total_employees();
```

The screenshot shows a database management tool interface with the following components:

- Header:** Server: 127.0.0.1 > Database: university_db
- Navigation Bar:** Structure, SQL, Search, Query, Export, Import, Operations
- Buttons:** Show query box
- Warning:** Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.
- Status Bar:** Showing rows 0 - 0 (1 total, Query took 0.0006 seconds.)
- Code Editor:** CALL total_employees();
[Edit inline] [Edit] [Create PHP code]
- Table Controls:** ☐ Show all | Number of rows: 25 | Filter rows: Search this table
- Table Data:**

total_employees
5

Lab 2 : Create a PL/SQL block that calculates the total sales from an orders table.

```
- DELIMITER $$  
CREATE PROCEDURE total_sales_amount()  
BEGIN  
DECLARE total_sales DECIMAL(10,2);  
SELECT SUM(order_amount) INTO total_sales  
FROM orders;  
SELECT total_sales AS total_sales;  
END $$  
DELIMITER ;  
CALL total_sales_amount();
```

The screenshot shows a database management tool interface. At the top, the breadcrumb navigation indicates the path: Server: 127.0.0.1 » Database: university_db » Table: orders. Below this is a toolbar with buttons for Browse, Structure, SQL, Search, Insert, Export, and Import. A 'Show query box' button is located below the toolbar. A warning message states: 'Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete feature'. A green status bar indicates: 'Showing rows 0 - 0 (1 total, Query took 0.0008 seconds.)'. Below this, the executed SQL code is displayed: `CALL total_sales_amount();`. There are links for '[Edit inline]', '[Edit]', and '[Create PHP code]'. A control bar includes a 'Show all' checkbox, a 'Number of rows' dropdown set to 25, and a 'Filter rows' search box. An 'Extra options' button is also present. At the bottom, a table with one row is shown, with the column header 'total_sales' and the value '15000.00'.

total_sales
15000.00

17. PL/SQL Control Structures

Lab 1 : Write a PL/SQL block using an IF-THEN condition to check the department of an employee.

```
- DELIMITER $$
CREATE PROCEDURE check_department(IN p_emp_id INT)
BEGIN
  DECLARE v_department VARCHAR(50);
  SELECT department
  INTO v_department
  FROM employees
  WHERE emp_id = p_emp_id;
  IF v_department = 'IT' THEN
    SELECT 'Employee belongs to IT Department' AS Result;
  ELSE
    SELECT 'Employee does not belong to IT Department' AS Result;
  END IF;
END $$
DELIMITER ;
CALL check_department(2);
```

The screenshot shows a database management tool interface with a top toolbar containing icons for Structure, SQL, Search, Query, Export, Import, and Operations. Below the toolbar is a 'Show query box' button. A warning message states: 'Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are disabled.' A green status bar indicates: 'Showing rows 0 - 0 (1 total, Query took 0.0007 seconds.)'. The SQL editor displays the executed query: `CALL check_department(1);`. Below the editor are links for '[Edit inline]', '[Edit]', and '[Create PHP code]'. A control bar includes a 'Show all' checkbox, a 'Number of rows' dropdown set to 25, and a 'Filter rows' search box. An 'Extra options' button is located below the control bar. The 'Result' section shows the output: 'Employee belongs to IT Department'.

Lab 2 : Use a FOR LOOP to iterate through employee records and display their names.

```
- DELIMITER $$  
CREATE PROCEDURE show_employee_names()  
BEGIN  
    DECLARE done INT DEFAULT 0;  
    DECLARE v_name VARCHAR(100);  
    DECLARE emp_cursor CURSOR FOR  
        SELECT emp_name FROM employees;  
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;  
    OPEN emp_cursor;  
read_loop: LOOP  
        FETCH emp_cursor INTO v_name;  
    IF done = 1 THEN  
        LEAVE read_loop;  
    END IF;  
    SELECT v_name AS Employee_Name;  
END LOOP;  
CLOSE emp_cursor;  
END$$  
DELIMITER ;
```

The screenshot shows a MySQL database interface with the following components:

- Server:** 127.0.0.1
- Database:** university_db
- Navigation Bar:** Structure, SQL, Search, Query, Export, Import, Operations
- Status Bar:** A green message box stating: "Your SQL query has been executed successfully. 1 row affected by the last statement inside the procedure."
- Query Editor:** Contains the command: `CALL `show_employee_names`();`
- Execution Results:** A window titled "Execution results of routine `show\_employee\_names`" displays the output of the procedure. It shows five rows, each with the column header "Employee_Name" and a corresponding name value.

Employee_Name
ayush
dhaval
jay
meet
tilak

18. SQL Cursors

Lab 1 : Write a PL/SQL block using an explicit cursor to retrieve and display employee details.

```
- DELIMITER $$
CREATE PROCEDURE show_employee_details()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_id INT;
    DECLARE v_name VARCHAR(100);
    DECLARE v_dept VARCHAR(50);
    DECLARE emp_cursor CURSOR FOR
        SELECT emp_id, emp_name, department FROM employees;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    OPEN emp_cursor;
    emp_loop: LOOP
        FETCH emp_cursor INTO v_id, v_name, v_dept;
        IF done = 1 THEN
            LEAVE emp_loop;
        END IF;
        SELECT v_id AS Emp_ID,
            v_name AS Emp_Name,
            v_dept AS Department;
    END LOOP;
    CLOSE emp_cursor;
END$$
DELIMITER ;
```

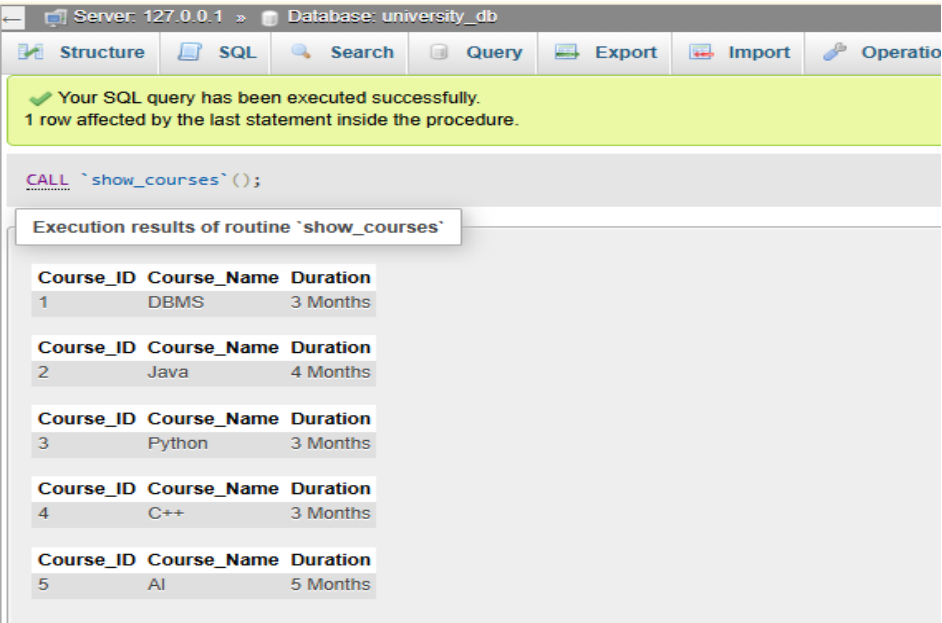
The screenshot shows a database client window with the following elements:

- Top bar: Server: 127.0.0.1 > Database: university_db
- Navigation tabs: Structure, SQL, Search, Query, Export, Import, Operations
- Message bar: A green bar indicating "Your SQL query has been executed successfully. 1 row affected by the last statement inside the procedure."
- SQL Editor: Contains the command `CALL `show_employee_details`();`
- Results Panel: Titled "Execution results of routine `show\_employee\_details`", it displays five rows of employee data in a table format.

Emp_ID	Emp_Name	Department
1	ayush	IT
2	dhaval	HR
3	jay	Finance
4	meet	Manager
5	tilak	IT

Lab 2 : Create a cursor to retrieve all courses and display them one by one.

```
- DELIMITER $$
CREATE PROCEDURE show_courses()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_id INT;
    DECLARE v_name VARCHAR(100);
    DECLARE v_duration VARCHAR(50);
    DECLARE course_cursor CURSOR FOR
        SELECT course_id, course_name, course_duration FROM courses;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    OPEN course_cursor;
    course_loop: LOOP
        FETCH course_cursor INTO v_id, v_name, v_duration;
        IF done = 1 THEN
            LEAVE course_loop;
        END IF;
        SELECT v_id AS Course_ID, v_name AS Course_Name, v_duration
AS Duration;
    END LOOP;
    CLOSE course_cursor;
END$$
DELIMITER ;
```



Server: 127.0.0.1 » Database: university_db

Structure SQL Search Query Export Import Operations

✓ Your SQL query has been executed successfully.
1 row affected by the last statement inside the procedure.

CALL `show\_courses`();

Execution results of routine `show\_courses`

Course_ID	Course_Name	Duration
1	DBMS	3 Months
2	Java	4 Months
3	Python	3 Months
4	C++	3 Months
5	AI	5 Months

19. Rollback and Commit Savepoint

Lab 1 : Perform a transaction where you create a savepoint, insert records, then rollback to the savepoint.

- START TRANSACTION;
INSERT INTO courses (course_name, course_duration)
VALUES ('Cloud', '4 Months');
SAVEPOINT sp1;
INSERT INTO courses (course_name, course_duration)
VALUES ('DevOps', '5 Months');
ROLLBACK TO sp1;
COMMIT;

The screenshot shows a database management interface with the following components:

- Header:** Server: 127.0.0.1 » Database: university_db » Table: courses
- Navigation Bar:** Browse, Structure, SQL, Search, Insert, Export, Import
- Buttons:** Show query box
- Status Bar:** ✓ Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)
- Query Editor:** `SELECT * FROM `courses`;`
- Options:** ☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
- Display Controls:** ☐ Show all | Number of rows: 25 | Filter rows: Search this table
- Extra options:** Extra options
- Table View:**

	course_id	course_name	course_duration
☐	1	Cloud	4 Months
- Actions:** Edit, Copy, Delete

Lab 2 : Commit part of a transaction after using a savepoint and then rollback the remaining changes.

- START TRANSACTION;
INSERT INTO courses (course_name, course_duration)
VALUES ('Java Advanced', '5 Months');
SAVEPOINT sp2;
INSERT INTO courses (course_name, course_duration)
VALUES ('Spring Boot', '4 Months');
COMMIT;
START TRANSACTION;
INSERT INTO courses (course_name, course_duration)
VALUES ('Microservices', '6 Months');
ROLLBACK;

Server: 127.0.0.1 » Database: university_db » Table: courses

Browse Structure SQL Search Insert Export Import

Show query box

✓ Showing rows 0 - 2 (3 total, Query took 0.0004 seconds.)

`SELECT * FROM `courses`;`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 ▼ Filter rows: Search this table Sort by key:

Extra options

			course_id	course_name	course_duration	
☐	Edit	Copy	Delete	1	Cloud	4 Months
☐	Edit	Copy	Delete	3	Java Advanced	5 Months
☐	Edit	Copy	Delete	4	Spring Boot	4 Months